Chapter 7. Serving Models and Architectures

As we think about how recommendation systems utilize the available data to learn and eventually serve recommendations, it's crucial to describe how the pieces fit together. The combination of the data flow and the jointly available data for learning is called the *architecture*. More formally, the architecture is the connections and interactions of the system or network of services; for data applications, the architecture also includes the available features and objective functions for each subsystem. Defining the architecture typically involves identifying components or individual services, defining the relationships and dependencies among those components, and specifying the protocols or interfaces through which they will communicate.

In this chapter, we'll spell out some of the most popular and important architectures for recommendation systems.

Architectures by Recommendation Structure

We have returned several times to the concept of collector, ranker, and server, and we've seen that they may be regarded via two paradigms: the online and the offline modes. Further, we've seen how many of the components in Chapter 6 satisfy some of the core requirements of these functions.

Designing large systems like these requires several architectural considerations. In this section, we will demonstrate how these concepts are adapted based on the type of recommendation system you are building.

We'll compare a mostly standard item-to-user recommendation system, a query-based recommendation system, context-based recommendations, and sequence-based recommendations.

Item-to-User Recommendations

We'll start by describing the architecture of the system we've been building in the book thus far. As proposed in Chapter 4, we built the collector offline to ingest and process our recommendations. We utilize representations to encode relationships between items, users, or user-item pairs.

The online collector takes the request, usually in the form of a user ID, and finds a neighborhood of items in this representation space to pass along to the ranker. Those items are filtered when appropriate and sent for scoring.

The offline ranker learns the relevant features for scoring and ranking, training on the historical data. It then uses this model and, in some cases, item features as well for inference.

In the case of recommendation systems, this inference computes the scores associated to each item in the set of potential recommendations. We usually sort by this score, which you'll learn more about in Part III. Finally, we integrate a final round of ordering based on some business logic (described in Chapter 14). This last step is part of the serving, where we impose requirements like test criteria or recommendation diversity requirements.

Figure 7-1 is an excellent overview of the retrieval, ranking, and serving structure, although it depicts four stages and uses slightly different terminology. In this book, we combine the filtering stage shown here into retrieval.

Query-Based Recommendations

To start off our process, we want to make a query. The most obvious example of a query is a text query as in text-based search engines; however,

queries may be more general! For example, you may wish to allow search-by-image or search-by-tag options. Note that an important type of query-based recommender uses an *implicit* query: the user is providing a search query via UI choices or by behaviors. While these systems are quite similar in overall structure to the item-to-user systems, let's discover how to modify them to fit our use case.

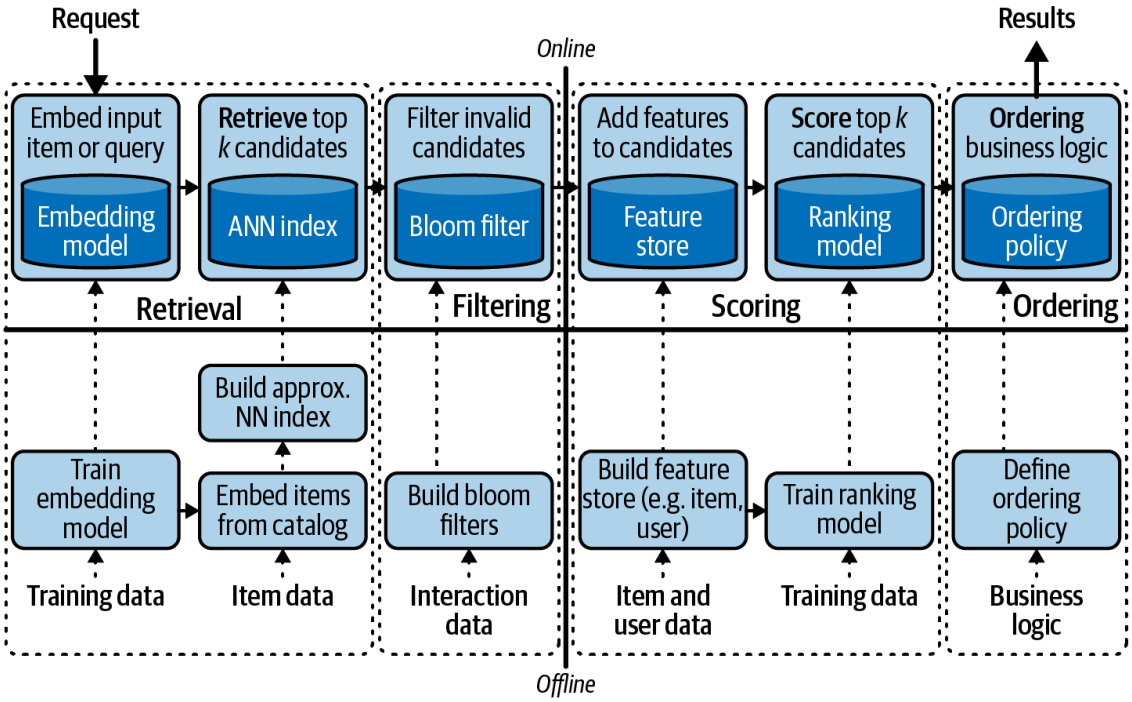

Figure 7-1. A four-stage recommendation system (adapted from an image by Karl Higley and Even Oldridge)

We want to integrate more context about the query into the first step of the request. Note that we don't want to throw out the user-item matching components of this system. Even though the user is performing a search, personalizing the recommendations based on their taste is useful. Instead, we need to utilize the query as well; later we will discuss various technical strategies, but a simple summary for now is to also generate an embedding for the query. Note that the query is like an item or a user but is sufficiently different.

Some strategies might include similarity between the query and items, or co-occurrence of the query and items. Either way, we now have a query representation and user representation, and we want to utilize both for our recommendation. One simple approach is to use the query representation for retrieval, but during the scoring, score via both query-item and

user-item, combining them via a multiobjective loss. Another approach is to use the user for retrieval and then the query for filtering.

DIFFERENT EMBEDDINGS

Unfortunately, while we'd love the same embedding space (for nearest-neighbors lookup) to work well for our queries and our documents (items, etc.), this is often not the case. The simplest example is something like asking questions and hoping to find relevant Wikipedia articles. This problem is often referred to as the queries being "out of distribution" from the documents.

Wikipedia articles are written in a declarative informative article style, whereas questions are often brief and casual. If you were to use an embedding model focused on capturing semantic meaning, you'd naively expect the queries to be located in significantly different subspaces than the articles. This means that your distance computations will be affected. This is often *not* a huge problem because you retrieve via relative distances, and you can hope that the shared subspaces are enough to provide a good retrieval. However, it can be hard to predict when these perform poorly.

The best practice is to carefully examine the embeddings on common queries and on target results. These problems can be *especially* bad on implicit queries like a series of actions taken at a particular time of day to look up food recommendations. In this case, we expect the queries to be wildly different from the documents.

Context-Based Recommendations

A context is quite similar to a query but tends to be more obviously feature based and frequently less similar to the items/users distributions. *Context* is usually the term used to represent exogenous features to the system that may have an effect on the system—i.e., auxiliary information such as time, weather, or location. Context-based recommendation is similar to query based in that context is an additional signal that the system needs to consider during recommendation, but more often than not, the query should dominate the signal for recommendation, whereas the context should not.

Let's take a simple example of ordering food. A query for a food-delivery recommendation system would look like *Mexican food*; this is an extremely important signal from the user looking for burritos or quesadillas of how the recommendations should look. A context for a food-delivery recommendation system would look like *it's almost lunchtime*. This signal is useful but may not outweigh user personalization. Putting hard-and-fast rules on this weighting can be difficult, so usually we don't, and instead we learn parameters via experimentation.

Context features fit into the architecture similar to the way queries do, via learned weightings as part of the objective function. Your model will learn a representation between context features and items, and then add that affinity into the rest of the pipeline. Again, you can make use of this early in the retrieval, later in the ranking, or even during the serving step.

Sequence-Based Recommendations

Sequence-based recommendations build on context-based recommendations but with a specific type of context. Sequential recommendations are based on the idea that the recent items the user has been exposed to should have a significant influence on the recommendations. A common example here is a music-streaming service, as the last few songs that have been played can significantly inform what the user might want to hear next. To ensure that this *autoregressive*, or sequentially predictive, set of features has an influence on recommendations, we can treat each item in the sequence as a weighted context for the recommendation.

Usually, the item-item representation similarities are weighted to provide a collection of recommendations, and various strategies are used for combining these. In this case, we normally expect the user to be of high importance in the recommendations, but the sequence is also of high importance. One simple model is to think of the sequence of items as a sequence of tokens, and form a single embedding for that sequence—as in NLP applications. This embedding can be used as the context in a context-based recommendation architecture.

NAIVE SEQUENCE EMBEDDINGS

The combinatorics of one-embedding-per-sequence explode in cardinality; the number of potential items in each sequential slot is very large, and each item in the sequence multiplies those possibilities together. Imagine, for example, five-word sequences, where the number of possibilities for each item is close to the size of the English lexicon, and thus it would be that size to the fifth power. We provide simple strategies for dealing with this in Chapter 17.

Why Bother with Extra Features?

Sometimes it is useful to step back and ask if a new technology is actually worth caring about. So far in this section, we've introduced four new paradigms for thinking about a recommender problem. That level of detail may seem surprising and potentially even unnecessary.

One of the core reasons that things like context- and query-based recommendations become relevant is to deal with some of the issues mentioned before around sparsity and cold starting. Sparsity makes things that aren't cold seem cold via the learner's underexposure to them, but true cold starting also exists because of new items being added to catalogs with high frequency in most applications. We will address cold starting in detail, but for now, suffice it to say that one strategy for warm starting is to use other features that *are* available even in this regime.

In applications of ML that are explicitly feature based, we rarely battle the cold-start problem to such a degree, because at inference time we're confident that the model parameters useful for prediction are well aligned with those features that are available. In this way, feature-included recommendation systems are bootstrapping from a potentially weaker learner that has more guaranteed performance via always-available features.

The second analogy that the previous architectures are reflecting is that of boosting. Boosted models operate via the observation that ensembles of weaker learners can reach better performance. Here we are asking for some additional features to help these networks ensemble with weak learners, to boost their performance.

Encoder Architectures and Cold Starting

The previous problem framings of various types of recommendation problems point out four model architectures, each fitting into our general framework of collector, ranker, and server. With this understanding, let's discuss in a bit more detail how model architecture can become intertwined with serving architecture. In particular, we also need to discuss feature encoders.

The key opportunity from encoder-augmented systems is that for users, items, or contexts without much data, we can still form embeddings on the fly. Recall from before that our embeddings make the rest of our system possible, but cold-starting recommendations is a huge challenge.

The *two-towers architecture*—or dual-encoder networks—introduced in "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations" by Xinyang Yi et al. is shown in Figure 7-2 explicit model architecture is aimed at prioritizing features of both the user and items when building a scoring model for a recommendation system. We'll see a lot more discussion of matrix factorization (MF), which is a kind of latent collaborative filtering (CF) derived from the user-item matrix and some linear algebraic algorithms. In the preceding section, we explained why additional features matter. Adding these *side-car* features into an MF paradigm is possible and has shown to be successful—for example, applications CF for implicit feedback, factorization machines, and SVDFeature. However, in this model we will take a more direct approach.

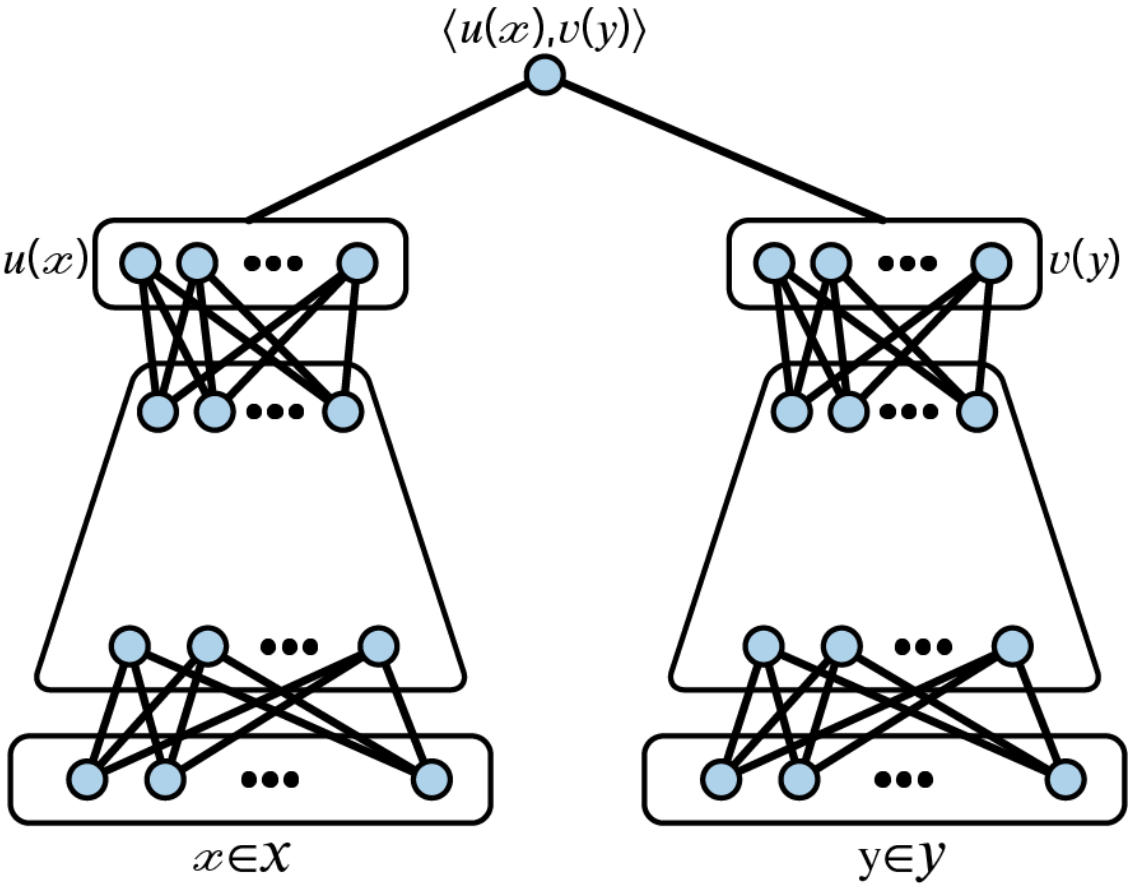

Figure 7-2. The two towers responsible for the two embeddings

In this architecture, we take the left tower to be responsible for items and the right tower to be responsible for the user and, when appropriate, context. These two tower architectures are inspired by the NLP literature and, in particular, "Learning Text Similarity with Siamese Recurrent Networks" by Paul Neculoiu et al.

Let's detail how this model architecture is applied to recommending videos on YouTube. For a full overview of where this architecture was first introduced, see "Deep Neural Networks for YouTube Recommendations" by Paul Covington et al. Training labels will be given by clicks, but with an additional regression feature $r_i \in 0, 1\}$, where the minimum value corresponds to a click but trivial watch time, and the maximum of the range corresponds to a full watch.

As we've mentioned, this model architecture will explicitly include features from both user and items. The video features will consist of categorical and continuous features, like `VideoId`, `ChannelId`, `VideoTopic`, and so on. An embedding layer is used for many of the categorical features to

move to dense representations. The user features include watch histories via bag of words and standard user features.

This model structure combines many of the ideas you've seen before but has relevant takeaways for our system architecture. First is the idea of sequential training. Each *temporal batch* of samples should be trained in sequence to ensure that model drift is shown to the model; we will discuss prequential datasets in "Prequential validation". Next, we present an important idea for the productionizing of these kinds of models: encoders.

In these models, we have feature encoders as the early layers in both towers, and when we move to inference, we will still need these encoders. When performing the online recommendations, we will be given `UserId` and `VideoId` and will first need to collect their features. As discussed in "Feature Stores", the feature store will be useful in getting these raw features, but we need to also encode the features into the dense representations necessary for inference. This is something that can be stored in the feature store for known entities, but for unknown entities we will need to do the feature embedding at inference time.

Encoding layers serve as a simple model for mapping a collection of features to a dense representation. When fitting encoding layers as the first step in a neural network, the common strategy is to take the first k layers and reuse them as an encoder model. More specifically, if $\mathscr{L}^i, 0 \leq i \leq k$ are the layers responsible for feature encoding, call $Emb\left(\widehat{V}\right) = \mathscr{L}^k\left(\mathscr{L}^{k-1}\left(...\mathscr{L}^0\left(\widehat{V}\right)\right)\right)$ the function that maps a feature vector $\widehat{V}$ to its dense representation.

In our previous system architecture, we would include this encoder as part of the fast layer, after receiving features from the feature store. It's also important to note that we would still want to utilize vector search; these feature embedding layers are used upstream of the vector search and nearest neighbor searches.

ENCODER AS A SERVICE

Encoders and retrieval are a key part of the multistage recommendation pipeline. We've spoken briefly about the latent spaces in question (for more details, see "Latent Spaces"), and we've alluded to an *encoder*. Briefly, an encoder is the model that converts users, items, queries, etc., into the latent space in which you'll perform nearest-neighbors search. These models can be trained via a variety of processes, many of which will be discussed later, but it's important to discuss where they live once trained.

Encoders are often simple API endpoints that take the content to be embedded and return a vector (a list of floats). Encoders often work at the batch layer to encode all the documents/items that will be retrieved, but they must *also* be connected to the real-time layer to encode the queries as they come in. A common pattern is to set up a batch endpoint and a single query endpoint to facilitate optimization for both modalities. These endpoints should be fast and highly available.

If you're working with text data, a good starting place is to use BERT or GPT-based embeddings. The easiest at this time are provided as a hosted service from OpenAI.

Deployment

Like many ML applications, the final output of a recommendation system is itself a small program that runs continuously and exposes an API to interact with it; batch recommendations are often a powerful place to start, performing all the necessary recommendations ahead of time. Throughout this chapter, we've seen the pieces embedded in our backend system, but now we will discuss the components closer to the user.

In our relatively general architecture, the server is responsible for handing over the recommendations, after all the work that comes before, and should adhere to a preset schema. But what does this deployment look like?

Models as APIs

Let's discuss two systems architectures that might be appropriate for serving your models in production: microservice and monolith.

In web applications, this dichotomy is well covered from many perspectives and special use cases. As ML engineers, data scientists, and potentially data platform engineers, it's not necessary to dig deep into this area, but it's essential to know the basics:

Microservice architectures

Each component of the pipeline should be its own small program with a clear API and output schema. Composing these API calls allows for flexible and predictable pipelines.

Monolithic architectures

One application should contain all the necessary logic and components for model predictions. Keeping the application self-contained means fewer interfaces that need to be kept aligned and fewer rabbit holes to hunt around in when a location in your pipeline is being starved.

Whatever you choose as your strategy, you'll need to make a few decisions:

How large is the necessary application?

If your application will need fast access to large datasets at inference time, you'll need to think carefully about memory requirements.

What access does your application need?

We've previously discussed using technologies like bloom filters and feature stores. These resources may be tightly coupled to your application (by building them in memory in the application) or may be an API call away. Make sure your deployment accounts for these relationships.

Should your model be deployed to a single node or a cluster?

For some model types, even at the inference step we wish to utilize distributed computing. This will require additional configuration to allow for fast parallelization.

How much replication do you need?

Horizontal scaling allows you to have multiple copies of the same service running simultaneously to reduce the demand on any particular instance. This is important for ensuring availability and performance. As we horizontally scale, each service can operate independently, and various strategies exist for coordinating these services and an API request. Each replica is usually its own containerized application, and these APIs like CoreOS and Kubernetes are used to manage these. The requests themselves must also be balanced to the different replicas via something like nginx.

What are the relevant APIs that are exposed?

Each application in the stack should have a clear set of exposed schemas and an explicit communication about the types of other applications that may call to the APIs.

Spinning Up a Model Service

So what can you use to get your model into an application? A variety of frameworks for application development are useful; some of the most popular in Python are Flask, FastAPI, and Django. Each has different advantages, but we'll discuss FastAPI here.

FastAPI is a targeted framework for API applications, making it especially well fit for serving ML models. It calls itself an asynchronous server gateway interface (ASGI) framework, and its specificity grants a ton of simplicity.

Let's take a simple example of turning a fit torch model into a service with the FastAPI framework. First, let's utilize an artifact store to pull down our fit model. Here we are using the Weights & Biases artifact store:

```
import wandb, torch
run = wandb.init(project=Prod_model, job_type="inference")

model_dir = run.use_artifact(
                'bryan-wandb/recsys-torch/model:latest',
                type='model'
).download()

model = torch.load(model_dir)
model.eval(user_id)
```

This looks just like your notebook workflow, so let's see how easy it is to integrate this with FastAPI:

```
from fastapi import FastAPI # FastAPI code

import wandb, torch

app = FastAPI() # FastAPI code

run = wandb.init(project=Prod_model, job_type="inference")

model_dir = run.use_artifact(
        'bryan-wandb/recsys-torch/model:latest',
        type='model'
).download()

model = torch.load(model_dir)

@app.get("/recommendations/{user_id}") # FastAPI code
def make_recs_for_user(user_id: int): # FastAPI code
                endpoint_name = 'make_recs_for_user_v0'
                logger.info(
                        "{'type': 'recommendation_request',"
                        f"'arguments': {'user_id': {user_id}},"
                        f"'response': {None}},",
                        f"'endpoint_name': {endpoint_name}"
                )
                recommendation = model.eval(user_id)
                logger.log(
                        "{'type': 'model_inference',"
```

```
                        f"'arguments': {'user_id': {user_id}},"
                        f"'response': {recommendation}},"
                        f"'endpoint_name': {endpoint_name}"
                    )
    return { # FastAPI code
                        "user_id": user_id,
                        "endpoint_name": endpoint_name,
                        "recommendation": recommendation
                    }
```

I hope you share my enthusiasm that we now have a model as a service in five additional lines of code. While this scenario includes simple examples of logging, we'll discuss logging in greater detail later in this chapter to help you improve observability in your applications.

Workflow Orchestration

The other component necessary for your deployed system is workflow orchestration. The model service is responsible for receiving requests and serving results, but many system components need to be in place for this service to do anything of use. These workflows have several components, so we will discuss them in sequence: containerization, scheduling, and CI/CD.

Containerization

We've discussed how to put together a simple service that can return the results, and we suggested using FastAPI; however, the question of environments is now relevant. When executing Python code, it is important to keep the environment consistent if not identical. FastAPI is a library for designing the interfaces; Docker is the software that manages the environment that code runs in. It's common to hear Docker described as a container or containerization tool: this is because you load a bunch of apps—or executable components of code—into one shared environment.

We have a few subtle things to note at this point. The meaning of *environment* encapsulates both the Python environment of package dependencies and the larger environment, including the operating system or GPU

drivers. The environment is usually initialized from a predetermined *image* that installs the most basic aspects of what you'll need access to and in many cases is less variable across services to promote consistency and standardization. Finally, the container is usually equipped with a list of infrastructure code necessary to work wherever it is to be deployed.

In practice, you specify details of the Python environment via your *requirements* file, which consists of a list of Python packages. Note that some library dependencies are outside Python and will require additional configuration mechanisms. The operating system and drivers are usually built as part of a base image; you can find these on DockerHub or similar. Finally, *infrastructure as code* is a paradigm wherein you write code to orchestrate the necessary steps in getting your container configured to run in the infrastructure it will be deployed into. Dockerfile and Docker Compose are specific to the Docker container interfacing with infrastructure, but you can further generalize these concepts to include other details of the infrastructure. This infrastructure as code begins to encapsulate provisioning of resources in your cloud, setting up open ports for network communication, access control via security roles, and more. A common way to write this code is in Terraform. This book doesn't dive into infrastructure specification, but infrastructure as code is becoming a more important tool to the ML practitioner. Many companies are beginning to attempt to simplify these aspects of training and deploying systems including Weights & Biases or Modal.

Scheduling

Two paradigms exist for scheduling jobs: cron and triggers. Later we'll talk more about the continuous training loop and active learning processes, but upstream of those is your ML workflow. ML workflows are a set of ordered steps necessary to prepare your model for inference. We've introduced our notion of collector, ranker, and server, which are organized into a sequence of stages for recommendation systems—but these are the three coarsest elements of the system topology.

In ML systems, we frequently assume that there's an upstream stage of the workflow that corresponds to data transformations, as discussed in

Chapter 6. Wherever that stage takes place, the output of those transformations results in our vector store—and potentially the additional feature stores. The handoff between those steps and the next steps in your workflow are the result of a job scheduler. As mentioned previously, tools like Dagster and Airflow can run sequences of jobs with dependent assets. These kinds of tools are needed to orchestrate the transitions and to ensure that they're timely.

Cron refers to a time schedule where a workflow should begin—for example, hourly at the top of the hour or four times a day. *Triggers* refers to the instigation of a job run when another event has taken place—for example, if an endpoint receives a request, or a set of data gets a new version, or a limit of responses is exceeded. These are meant to capture more ad hoc relationships between the next job stage and the trigger. Both paradigms are very important.

CI/CD

Your workflow execution system is the backbone of your ML systems, often the bridge between the data collection process, the training process, and the deployment process. Modern workflow execution systems also include automatic validation and tracking so that you can audit the steps on the way to production.

Continuous integration (CI) is a term taken from software engineering to enforce a set of checks on new code in order to accelerate the development process. In traditional software engineering, this comprises automating unit and integration testing, usually run after checking the code into version control. For ML systems, CI may mean running test scripts against the model, checking the typed output of data transformations, or running validation sets through the model and benchmarking the performance against previous models.

Continuous deployment (CD) is also a term popularized in software engineering to refer to automating the process of pushing new packaged code into an existing system. In software engineering, deploying code when it has passed the relevant checks speeds development and reduces the risk

of stale systems. In ML, CD can involve strategies like automatically deploying your new model behind a service endpoint in shadow (which we'll discuss in "Shadowing") to test that it works as expected under live traffic. It could also mean deploying a model behind a very small allocation of an A/B test or multiarm bandit treatment to begin to measure effects on target outcomes. CD usually requires effective triggering by the requirements it has to satisfy before being pushed. It's common to hear CD utilizing a model registry, where you house and index variations on your model.

Alerting and Monitoring

Alerting and monitoring take a lot of their inspiration from the DevOps world for software engineering. Here are some high-level principles that will guide our thinking:

- Clearly defined schemas and priors
- Observability

Schemas and Priors

When designing software systems, you almost always have expectations about how the components fit together. Just as you anticipate the input and output to functions when writing code, in software systems you anticipate these at each interface. This is relevant not only for microservice architectures; even in a monolith architecture, components of the system need to work together and often have boundaries between their defining responsibilities.

Let's make this more concrete via an example. You've built a user-item latent space, a feature store for user features, a bloom filter for client avoids (things the client specifically tells you they don't want), and an experiment index that defines which of two models should be used for scoring. First let's examine the latent space; when provided a `user_id`, we need to look up its representation, and we already have some assumptions:

- The `user_id` provided will be of the correct type.
- The `user_id` will have a representation in our space.
- The representation returned will be of the correct type and shape.
- The component values of the representation vector will be in the appropriate domain. (*The support of representations in your latent space may vary day to day*.)

From here, we need look up the k ANN, which incurs more assumptions:

- There are $\geq k$ vectors in our latent space.
- Those vectors adhere to the expected distributional behavior of the latent space.

While these seem like relatively straightforward applications of unit tests, canonizing these assumptions is important. Take the last assumption in both of the two services: how can you know the appropriate domain for the representation vectors? As part of your training procedure, you'll need to calculate this and then store it for access during the inference pipeline.

In the second case, when finding nearest neighbors in high-dimensional spaces, well-discussed difficulties arise in distributional uniformity, but this can mean particularly poor performance for recommendations. In practice, we have observed a spiky nature to the behavior of k-nearest neighbors in latent spaces, leading to difficult challenges downstream in ensuring diversity of recommendations. These distributions can be estimated as priors, and simple checks like KL divergence can be used online; we can estimate the average behavior of the embeddings and the difference between local geometries.

In both cases, collecting and logging the output of this information can provide a rich history of what is going on with your system. This can shorten debugging loops later if model performance is low in production.

Returning to the possibility of `user_id` lacking a representation in our space: this is precisely the cold-start problem! In that case, we need to transition over to a different prediction pipeline: perhaps user-feature-based, explore-exploit, or even hardcoded recommendations. In this set-

ting, we need to understand next steps when a schema condition is not met and then gracefully move forward.

Integration Tests

Let's consider one higher-level challenge that might emerge in a system like this at the level of integration. Some refer to these issues as *entanglement*.

You've learned through experimentation that you should find $k = 20$ ANNs in the item space for a user to get good recommendations. You make a call to your representation space, get your 20 items, and pass them onto the filtering step. However, this user is quite picky; they have previously made many restrictions on their account about the kind of recommendations they allow: no shoes, no dresses, no jeans, no hats, no handbags—what's a struggling recommendation system to do?

Naively, if you take the 20 neighbors and pass them into the bloom, you're likely to be left with nothing! You can approach this challenge in two ways:

- Allow for a callback from the filter step to the retrieval (see "Predicate Pushdown")
- Build a user distribution and store that for access during retrieval

In the first approach, you give access to your filter step to call the retrieval step with a larger k until the requirements are satisfied after the bloom. Of course, this incurs significant slowdown as it requires multiple passes and ever-growing queries with redundancy! While this approach is simple, it requires building defensively and knowing ahead of time what may go wrong.

In the second approach, during training, you can sample from the user space to build estimates of the appropriate k for varying numbers of avoids by user. Then, giving access to a lookup of total avoids by user to the collector can help defend against this behavior.

OVER-RETRIEVAL

Sometimes people in information retrieval perform *over-retrieval* to mitigate issues of conflicting requirements from the search request, which can arise if the user makes a search and applies many filters simultaneously. This is applicable in recommendation systems as well.

If you retrieve only exactly the number of potential recommendations you need to serve to the user, downstream rules or poor personalization scores can sometimes cause a serious issue for serving up recommendations. This is why it is common to retrieve more items than you anticipate showing to the user.

Observability

Many tools in software engineering can assist with observability—understanding the *whys* of what's going on in the software stack. Because the systems we are building become quite distributed, the interfaces become critical monitoring points, but the paths also become complex.

Spans and traces

Common terms in this area are *spans* and *traces,* which refer to two dimensions of a call stack, illustrated in Figure 7-3. Given a collection of connected services, as in our preceding examples, an individual inference request will pass through some or all of those services in a sequence. The sequence of service requests is the *trace*. The potentially parallel time delays of each of these services is the *span*.

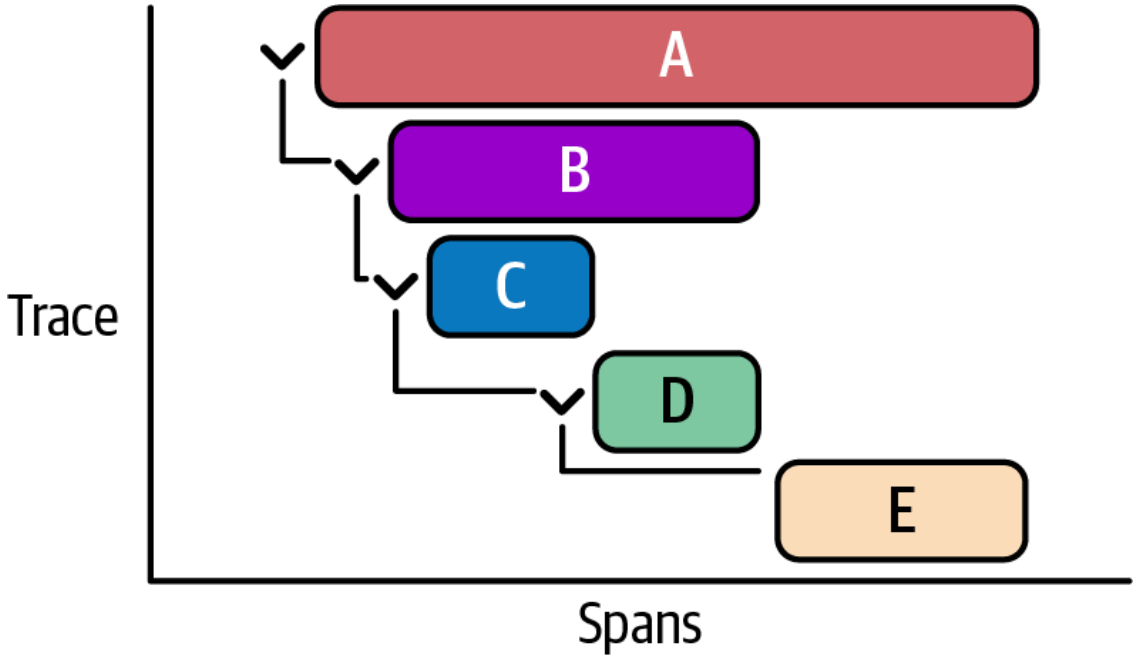

Figure 7-3. The spans of a trace

The graphical representation of spans usually demonstrates how the time for one service to respond comprises several other delays from other calls.

Observability enables you to see traces, spans, and logs in conjunction to appropriately diagnose the behavior of your system. In our example of utilizing a callback from the filter step to get more neighbors from the collector, we might see a slow response and wonder, "What has happened?" By viewing the spans and traces, we'd be able to see that the first call to the collector was as expected, then the filter step made a call to the collector, then another call to the collector, and so on, which built up a huge span for the filter step. Combining that view with logging would help us rapidly diagnose what might be happening.

Timeouts

In the preceding example, we had a long process that could lead to a very bad user experience. In most cases, we impose hard restrictions on how bad we let things get; these are called *timeouts*.

Usually, we have an upper bound on how long we're willing to wait for our inference response, so implementing timeouts aligns our system with these restrictions. It's important in these cases to have a *fallback*. In the

setting of recommendation systems, a fallback usually comprises things like the MPIR prepared such that it incurs minimal additional delay.

Evaluation in Production

If the previous section was about understanding what's coming into your model in production, this one might be summarized as what's coming out of your model in production. At a high level, evaluation in production can be thought of as extending all your model-validation techniques to the inference time. In particular, you are looking at *what the model actually is doing*!

On one hand, we already have tools to do this evaluation. You can use the same methods to evaluate performance as you do for training, but now on real observations streaming in. However, this process is not as obvious as we might first guess. Let's discuss some of the challenges.

Slow Feedback

Recommendation systems fundamentally are trying to lead to item selection, and in many cases, purchases. But if we step back and think more holistically about the purpose of integrating recommendation systems into businesses, it's to drive revenue. If you're an ecommerce shop, item selection and revenue may seem easily associated: a purchase leads to revenue, so good item recommendation leads to revenue. However, what about returns? Or even a harder question: is this revenue incremental? One challenge with recommendation systems is that it can be difficult to draw a causal arrow between any metric used to measure the performance of your models to the business-oriented KPIs.

We call this *slow feedback* because sometimes the loop from a recommendation to a meaningful metric and back to the recommender can take weeks or even longer. This is especially challenging when you want to run experiments to understand whether a new model should be rolled out. The length of the test may need to stretch quite a bit more to get meaningful results.

Usually, the team aligns on a proxy metric that the data scientists believe is a good estimator for the KPI, and that proxy metric is measured live. This approach has a huge variety of challenges, but it often suffices and provides motivation for more testing. Well-correlated proxies are often a great start to get directional information indicating where to take further iterations.

Model Metrics

So, what are the key metrics to track for your model in production? Given that we're looking at recommendation systems at inference time, we should seek to understand the following:

- Distribution of recommendation across categorical features
- Distribution of affinity scores
- Number of candidates
- Distribution of other ranking scores

As we discussed before, during the training process, we should be calculating broadly the ranges of our similarity scores in our latent space. Whether we are looking at high-level estimations or finer ones, we can use these distributions to get warning signals that something might be strange. Simply comparing the output of our model during inference, or over a set of inference requests, to these precompute distributions can be extremely helpful.

Comparing distributions can be a long topic, but one standard approach is computing *KL-divergence* between the observed distribution and the expected distribution from training. By computing KL divergence between these, we can understand how *surprising* the model's predictions are on a given day.

What we'd really like is to understand the receiver operating characteristic curve (ROC) of our model predictions with respect to one of our conversion types. However, this involves yet another integration to tie back to logging. Since our model API produces only the recommendation, we'll still need to tie into logging from the web application to understand out-

comes! To tie back in outcomes, we must join the model predictions with the logging output to get the evaluation labels, which can be done via log-parsing technologies (like Grafana, ELK, or Prometheus). We'll see more of this in Chapter 8.

RECEIVER OPERATING CHARACTERISTIC CURVE

If we assume that the relevance scores are estimating whether the item will be relevant to the user, this forms a binary classification problem. Utilizing these (normalized) scores, we can build an ROC to estimate over the distributions of queries when the relevance score begins to accurately predict a relevant item via retrieval history. This curve can thus be used to estimate parameters like necessary retrieval depth or even problematic queries.

Continuous Training and Deployment

It may feel like we're done with this story since we have models tracked and production monitoring in place, but rarely are we satisfied with set-it-and-forget-it model development. One important characteristic of ML products is that models frequently need to be updated to even be useful. Previously, we discussed model metrics and that sometimes performance in production might look different from our expectations based on the trained models' performance. This can be further exacerbated by model drift.

Model Drift

Model drift is the notion that the same model may exhibit different prediction behavior over time, merely due to changes in the data-generating process. A simple example is a time-series forecasting model. When you build a time-series forecasting model, the especially unique property that is essential for good performance is *autoregression*: the value of the function covaries with previous values of the function. We won't go into detail on time-series forecasting, but suffice it to say: your best hope of making a good forecast is to use up-to-date data! If you want to forecast stock

prices, you should always use the most recent prices as part of your predictions.

This simple example demonstrates how models may drift, and forecasting models are not so different from recommendation models—especially when considering the seasonal realities of many recommendation problems. A model that did well two weeks ago needs to be retrained with recent data to be expected to continue to perform well.

One criticism of a model that drifts is "that's the smoking gun of an overfit model," but in reality these models require a certain amount of overparameterization to be useful. In the context of recommendation systems, we've already seen that quirks like the Matthew effect have disastrous effects on the expected performance of a recommender model. If we don't consider things like new items in our recommender, we are doomed to fail. Models can drift for a variety of reasons, often coming down to exogenous factors in the generating process that may not be captured by the model.

One approach to dealing with and predicting stale models is to simulate these scenarios during training. If you suspect that the model goes stale mostly because of the distribution changing over time, you can employ sequential cross-validation—training on a contiguous period and testing on a subsequent period—but with a specified block of time delay. For example, if you think your model performance is going to decrease after two weeks because it's being trained on out-of-date observations, then during training you can purposely build your evaluation to incorporate a two-week delay before measuring performance. This is called *two-phase prediction comparison,* and by comparing the performances, you can estimate drift magnitudes to keep an eye out in production.

A wealth of statistical approaches can be used to rein in these differences. In lieu of a deep dive into variational modeling for variability and reliability for your predictions, we'll discuss continuous training and deployment and open this peanut with a sledge hammer.

Deployment Topologies

Let's consider a few structures for deploying models that will not only keep your models well in tune but also accommodate iteration, experimentation, and optimization.

Ensembles

Ensembles are a type of model structure in which multiple models are built, and the predictions from those models are pooled together in one of a variety of ways. While this notion of an ensemble is usually packaged into the model called for inference, you can generalize the idea to your deployment topology.

Let's take an example that builds on our previous discussion of prediction priors. If we have a collection of models with comparable performance on a task, we can deploy them in an ensemble, weighted by their deviation from the prior distributions of prediction that we've set before. This way, instead of having a simple yes/no filter on the output of your model's range, you can more smoothly transition potentially problematic predictions into more expected ones.

Another benefit of treating the ensemble as a deployment topology instead of only a model architecture is that you can *hot-swap* components of an ensemble as you make improvements in specific subdomains of your observation feature space. Take, for example, a life-time-value (LTV) model comprising three components: one that predicts well for new clients, another for activated clients, and a third for super-users. You may find that pooling via a voting mechanism performs the best on average, so you decide to implement a bagging approach. This works well, but later you find a better model for the new clients. By using the deployment topology for your ensemble, you can swap in the new model for the new clients and start comparing performance in your ensemble in production. This brings us to the next strategy, model comparison.

ENSEMBLE MODELING

Ensemble modeling is popular in all kinds of ML, built upon the simple notion that the mixture of expert opinions is strictly more effective than a single estimator. In fact, assume for a moment that you have M classifiers with error rate ϵ; then for an N class classification problem, your error would be $P(y \geq k) = \sum_{k}^{n} {}_{k}^{n} \epsilon^{k} (1-\epsilon)^{n-k}$, and the exciting part is that this is smaller than ϵ for all values less than 0.5!

Shadowing

Deploying two models, even for the same task, can be enormously informative. We call this *shadowing* when one model is "live" and the other is secretly also receiving all the requests and doing inference, and logging the results, of course. By shadowing traffic to the other model, you get the best expectations possible about how the model behaves before making your model live. This is especially useful when wanting to ensure that the prediction ranges align with expectation.

In software engineering and DevOps, there's a notion of *staging* for software. It's a hotly contested question of "how much of the real infrastructure should staging see," but shadowing is the staging of ML models. You can basically build a parallel pipeline for your entire infrastructure to connect for shadow models, or you can just put them both in the line of fire and have the request sent to both but use only one response. Shadowing is also crucial for implementing experimentation.

Experimentation

As good data scientists, we know that without a proper experimental framework, it's risky to advertise much about the performance of a feature or, in this case, model. Experimentation can be handled with shadowing by having a controller layer that is taking the incoming requests and orchestrating which of the deployed models to curry the response along. A simple A/B experimentation framework might ask for a randomization at every request, whereas something like a multiarmed bandit will require the controller layer to have notions of the reward function.

Experimentation is a deep topic that we don't have the knowledge or space to do adequate justice, but it's useful to know that this is where experimentation can fit into the larger deployment pipeline.

MODEL CASCADES

A really nice extension of the concepts of ensembling and shadowing is model cascading, illustrated in Figure 7-4. The simplified idea of a model cascade is that we use model confidence to create a conditional ensemble. In particular, given an inference request, the model provides a prediction with a confidence estimate; when the model confidence is high, that prediction is returned, but if the confidence is below a certain threshold, a downstream model is called and the ensemble is started. There's no reason to stop at two models; this method can be used to iteratively expand the number of ensemble layers for any number of models that in training show improved performance in an ensemble.

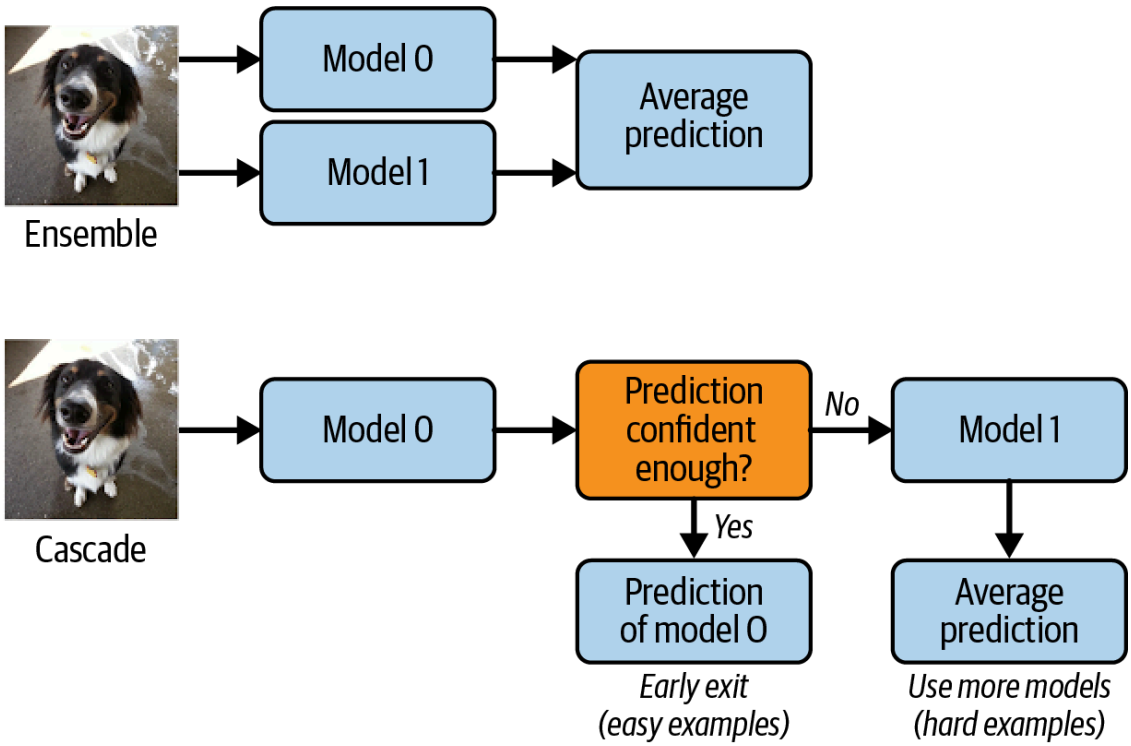

Figure 7-4. Ensembles versus cascades

Here are a few advantages of this approach:

- Better expected performance overall, as ensembles usually increase performance
- Ensemble performance with lower average computation time
- Especially better performance in out-of-sample scenarios

This method scales to larger pools of models and, while it may incur significant training efforts, finding the right ordering of the models can have

significant effects on model accuracy and latency.

The Evaluation Flywheel

By now, it's likely obvious that a production ML model is far from a static object. Production ML systems of any kind are subject to as many deployment concerns as a traditional software stack, in addition to the added challenge of dataset shift and new users/items. In this section, we'll look closely at the feedback loops introduced and understand how the components fit together to continuously improve our system—even with little input from a data scientist or ML engineer.

Daily Warm Starts

As we've now discussed several times, we need a connection between the continuous output of our model and retraining. The first simplest example of this is daily warm starts, which essentially ask us to utilize the new data seen each day in our system.

As might already be obvious, some of the recommendation models that show great success are quite large. Retraining some of them can be a massive undertaking, and simply *rerunning everything* each day is often infeasible. So, what can be done?

Let's ground this conversation in the user-user CF example that we've been sketching out; the first step was to build an embedding via our similarity definition. Let's recall:

$$\text{USim}_{A,B} = \frac{\sum_{x \in \mathscr{R}_{A,B}} (r_{A,x} - \overline{r}_A)(r_{B,x} - \overline{r}_B)}{\sqrt{\sum_{x \in \mathscr{R}_{A,B}} (r_{A,x} - \overline{r}_A)^2} \sqrt{\sum_{x \in \mathscr{R}_{A,B}} (r_{B,x} - \overline{r}_B)^2}}$$

Here we remember that the similarity between two users is dependent on the shared ratings and on each user's average rating.

On a given day, let's say
$\widetilde{X} = \{ \widetilde{x} \mid x \text{ was rated since yesterday by a user} \}$. Then we'd need to update our user similarities, but ideally we'd leave everything else the

same. To update the user's data, we see that all $\tilde{x}$ rated by two users, A and $B\bar{r}_A$ and $\bar{r}_B$, would need to change, but we could probably skip these updates in many cases where the number of ratings by those users was large. All in all, this means for each $\tilde{x}$, we should look up which users previously rated x and update the user similarity between them and the new rater.

This is a bit ad hoc, but for many methods you can utilize these tricks to reduce a full retraining. This would avoid a full batch retraining, via a fast layer. Other approaches exist, like building a separate model that can approximate recommendations for low-signal items. This can be done via feature models and can significantly reduce the complexity of these quick retrainings.

Lambda Architecture and Orchestration

On the more extreme end of the spectrum of these strategies is the lambda architecture; as discussed in Chapter 6, the lambda architecture seeks to have a much more frequent pipeline for adding new data into the system. The *speed* layer is responsible for working on small batches to perform the data transformations, and on model fitting to combine with the core model. As a reminder, many other aspects of the pipeline should also be updated during these fast layers, like the nearest neighbors graph, the feature store, and the filters.

Different components of the pipeline can require different investments to keep updated, so their schedules are an important consideration. You might be starting to notice that keeping all of these aspects in sync can be a bit challenging. If you have model training, model updating, feature store updates, redeployment, and new items/users all coming in on potentially different schedules, a *lot* of coordination may be necessary. This is where an *orchestration tool* can become relevant. A variety of approaches exist, but a few useful technologies here are GoCD, MetaFlow, and KubeFlow; the latter is more oriented at Kubernetes infrastructures. Another pipeline orchestration tool that can handle both batch and streaming pipelines is Apache Beam.

Generally, for ML deployment pipelines, we need to have a reliable core pipeline and the ability to keep the systems up to date as more data pours in. Orchestration systems usually define the topology of the systems, the relevant infrastructure configurations, and the mapping of the code artifacts needing to be run—not to mention the CRON schedules of when all these jobs need to run. Code as infrastructure is a popular paradigm that captures these goals as a mantra, so that even all this configuration itself is reproducible and automatable.

In all these orchestration considerations, there's a heavy overlap with containerization and *how* these steps may be deployed. Unfortunately, most of this discussion is beyond the scope of this book, but a simple overview is that containerized deployment with something like Docker is extremely helpful for ML services, and managing those deployments with various container management systems, like Kubernetes, is also popular.

Logging

Logging has come up several times already. Previously in this chapter, you saw that logging was important for ensuring that our system was behaving as expected. Let's discuss some best practices for logging and how they fit into our plans.

When we discussed traces and spans earlier, we were able to get a snapshot of the entire call stack of the services involved in responding to a request. Linking the services together to see the larger picture is incredibly useful, and when it comes to logging, gives us a hint as to how we should be orienting our thinking. Returning to our favorite RecSys architecture, we have the following:

- Collector receiving the request and looking up the embedding relevant to the user
- Computing ANN on items for that vector
- Applying filters via blooms to eliminate potential bad recommendations
- Augmenting features of the candidate items and user via the feature stores

- Scoring of candidates via the ranking model and estimating potential confidence
- Ordering and application of business logic or experimentation

Each of these elements has potential applications of logging, but let's now think about how to link them together. The relevant concept from microservices is correlation IDs; a *correlation ID* is simply an identifier that's passed along the call stack to ensure the ability to link everything later. As is likely obvious at this point, each of these services will be responsible for its own logging, but the services are almost always more useful in aggregate.

These days, Kafka is often used as the log-stream processor to listen for logs from all the services in your pipeline and to manage their processing and storing. Kafka relies on a message-based architecture; each service is a producer, and Kafka helps manage those messages to consumer channels. In terms of log management, the Kafka cluster receives all the logs in the relevant formats, hopefully augmented with correlation IDs, and sends them off to an ELK stack. The *ELK stack*—Elasticsearch, Logstash, Kibana—consists of a Logstash component to handle incoming log streams and apply structured processing, Elasticsearch to build search indices to the log store, and Kibana to add a UI and high-level dashboarding to the logging.

This stack of technologies is focused on ensuring that you have access and observability from your logs. Other technologies focus on other aspects, but what should you be logging?

Collector logs

Again, we wish to log during the following:

- Collector receiving the request and looking up the embedding relevant to the user
- Computing ANN on items for that vector

The collector receives a request, consisting in our simplest example of `user_id`, `requesting_timestamp`, and any augmenting keyword ele-

ments (kwargs) that might be required. A `correlation_id` should be passed along from the requester or generated at this step. A log with these basic keys should be fired, along with the timestamp of request received. A call is made to the embedding store, and the collector should log this request. Then the embedding store should log this request when received, along with the embedding store's response. Finally, the collector should log the response as it returns. This may feel like a lot of redundant information, but the explicit parameters included in the API calls become extremely useful when troubleshooting.

The collector now has the vector it will need to perform a vector search, so it will make a call to the ANN service. Logging this call, and any relevant logic in choosing the k for number of neighbors will be important, along with the ANN's received API request, the relevant state for computing ANN, and ANN's response. Back in the collector, logging that response and any potential data augmentation for downstream service requirements are the next steps.

At this point, at least six logs have been emitted—only reinforcing the need for a way to link these all together. In practice, you often have other relevant steps in your service that should be logged (e.g., checking that the distribution of distances in returned neighbors is appropriate for downstream ranking).

Note that if the embedding lookup was a miss, logging that miss is obviously important, as well as logging the subsequent request to the cold-start recommendation pipeline. The cold-start pipeline will incur additional logs.

Filtering and scoring

Now we need to monitor the following steps:

1. Applying filters via blooms to eliminate potential bad recommendations
2. Augmenting features to the candidate items and user via the feature stores

3. Scoring candidates via the ranking model, and potential confidence estimation

We should log the incoming request to the filtering service as well as the collection of filters we wish to apply. Additionally, as we search the blooms for each item and rule them in or out of the bloom, we should build up some structured logging of which items are caught in which filters and then log all this as a blob for later inspection. Responses and requests should be logged as part of feature augmentation—where we should log requests and responses to the feature store.

Also log the augmented features that end up attached to the item entities. This may seem redundant with the feature store itself, but understanding which features were added during a recommendation pipeline is *crucial* when looking back later to figure out why the pipeline might have behaved differently than anticipated.

At the time of scoring, the entire set of candidates should be logged with the features necessary for scoring and the output scores. It's extremely powerful to log this entire dataset, because training later can use these to get a better sense for real ranking sets. Finally, the response is passed to the next step with the ranked candidates and all their features.

Ordering

We have one more step to go, but it's an essential one: *ordering and application of business logic or experimentation*. This step is probably the most important logging step, because of how complicated and ad hoc the logic in this step can get.

If you have multiple intersecting business requirements implemented via filters at this step, while also integrating with experimentation, you can find yourself seriously struggling to unpack how reasonable expectations coming out of the ranker have turned into a mess by response time. Techniques like logging the incoming candidates, keyed to why they're eliminated, and the order of business rules applied will make reconstructing the behavior much more tractable.

Additionally, experimentation routing will likely be handled by another service, but the experiment ID seen in this step and the way that experiment assignment was utilized are the responsibility of the server. As we ship off the final recommendations, or decide to go another round, one last log of the state of the recommendation will ensure that app logs can be validated with responses.

NOTES ON FORMATTING

Structured logs are your friend. Implementing a data structure to hold the relevant data for your logs and then utilizing a log-formatter object will significantly reduce the difficulty in parsing and writing these logs. One often underappreciated feature of building message objects in code, and utilizing them as a running data structure throughout your call stack, is tight coupling between logs and app logic.

Tight coupling is often bemoaned in service-architecture discussions, but when that coupling is between your logs and your actual objects of execution, this saves you a lot of headaches. When changing the objects used for your service, instead of having an additional step to ensure the logs reflect that, you can propagate those changes through automatically by using the same objects in tandem with a log formatter.

These processes can also make good use of testing, to ensure that the objects your code cares about are visible in the logs, and these log-formatter objects can have enforced matching via unit tests. Finally, because we want to connect to downstream log parsing and log searching, it will be invaluable to have a clear relationship between the log stack and the application stack via object parameters and keys in the log data structure.

Active Learning

So far, we have discussed using updating data to train on a much more frequent schedule, and we've discussed how to provide good recommendations, even when the model hasn't seen enough data for those entities.

An additional opportunity for the feedback loop of recommendation and rating is active learning.

We won't be able to go deep into the topic, which is a large and active field of research, but we will discuss the core ideas in relation to recommendation systems. *Active learning* changes the learning paradigm a bit by suggesting that the learner should not only be passively collecting labeled (maybe implicit) observations but also attempting to mine relations and preferences from them. Active learning determines which data and observations would be most useful in improving model performance and then seeks out those labels. In the context of RecSys, we know that the Matthew effect is one of our biggest challenges, in that many potentially good matches for a user may be lacking enough or appropriate ratings to bubble to the top during the recommendations.

What if we employed a simple policy: every new item to the store gets recommended as a second option to the first 100 customers. Two outcomes would result:

- We would quickly establish data for our new item to help cold-start it.
- We would likely decrease the performance of our recommender.

In many cases, the second outcome is worth enduring to achieve the first, but when? And is this the right way to approach this problem? Active learning provides a methodical approach to these problems.

Another more specific advantage of active learning schemes is that you can broaden the distribution of observed data. In addition, to just cold-start items, we can use active learning to target broadening users' interests. This is usually framed as an uncertainty-reduction technique, as it can be used to improve the confidence in recommendations in a broader range of item categories. Here's a simple example: a user shops for only sci-fi books, so one day you show them a few extremely well-liked Westerns to see whether that user might be open to occasionally getting recommendations for Westerns. See "Propensity Weighting for Recommendation System Evaluation" for more details.

An active learning system is instrumented as a loss function inherited from the model it's trying to enhance—usually tied to uncertainty in some capacity—and it's attempting to minimize that loss. Given a model $\mathscr{M}$ trained on a set of observations and labels $\{x_i, y_i\}$, with loss $\mathscr{L}$, an active learner seeks to find a new observation, $\overline{x}$, such that if a label was obtained, $\overline{y}$, the loss would decrease via the model's training including this new pair. In particular, the goal is to approximate the marginal reduction in loss due to each possible new observation and find the observation that maximizes that reduction in the loss function:

$$\text{Argmax}_{\overline{x}} \left(\mathscr{L}\left(\mathscr{M}_{\{x_i, y_i\}}\right) - \mathscr{L}\left(\mathscr{M}_{\{x_i, y_i\} \cup \{\overline{x}\}}\right) \right)$$

The structure of an active learning system roughly follows these steps:

1. Estimate marginal decrease in loss due to obtaining one of a set of observations.
2. Select the observation with the largest effect.
3. *Query* the user; i.e., provide the recommendation to obtain a label.
4. Update the model.

It's probably clear that this paradigm requires a much faster training loop than our previous fast retraining schemes. Active learning can be instrumented in the same infrastructure as our other setups, or it can have its own mechanisms for integration into the pipeline.

Types of optimization

The optimization procedure carried out by an active learner in a recommendation system has two approaches: personalized and nonpersonalized. Because RecSys is all about personalization, it's no surprise that we would, in time, want to push the utility of our active learning further by integrating the great details we already know about users.

We can think of these two approaches as global loss minimization and local loss minimization. Active learning that isn't personalized tends to be about minimizing the loss over the entire system, not for only one user. (This split doesn't perfectly capture the ontology, but it's a useful mne-

monic). In practice, optimization methods are nuanced and sometimes utilize complicated algorithms and training procedures.

Let's talk through some factors to optimize for nonpersonalized active learning:

User rating variance

Consider which items have the largest variance in user ratings to try to get more data on those we find the most complicated in our observations.

Entropy

Consider the dispersion of ratings of a particular item across an ordinal feature. This is useful for understanding whether our set of ratings for an item is distributed uniformly at random.

Greedy extend

Measure which items seem to yield the worst performance in our current model; this attempts to improve our performance overall by collecting more data on the hardest items to recommend well.

Representatives or exemplars

Pick out items that are extremely representative of large groups of items; we can think of this as "If we have good labels for this, we have good labels for everything like this."

Popularity

Select items that the user is most likely to have experience with to maximize the likelihood that they'll give an opinion or rating.

Co-coverage

Attempt to amplify the ratings for frequently occurring pairs in the dataset; this strikes directly at the CF structure to maximize the utility of observations.

On the personalized side:

Binary prediction

To maximize the chances that the user can provide the requested rating, choose the items that the user is more likely to have experienced. This can be achieved via an MF on the binary ratings matrix.

Influence based

Estimate the influence of item ratings on the rating prediction of other items, and select the items with the largest influence. This attempts to directly measure the impact of a new item rating on the system.

Rating optimized

Obviously, there's an opportunity to simply use the best rating or best rating within a class to perform active learning queries, but this is precisely the standard strategy in recommendation systems to serve good recommendations.

User segmented

When available, use user segmentation and feature clusters within users to anticipate when users have opinions and preferences on an item by virtue of the user-similarity structure.

In general, a soft trade-off exists between active learning that's useful for maximally improving your model globally and active learning that's useful for maximizing the likelihood that a user can and will rate a particular item. Let's look at one particular example that uses both.

Application: User sign-up

One common hurdle to overcome in building recommendation systems is on-boarding new users. By definition, new users will be cold-starting with no ratings of any kind and will likely not expect great recommendations from the start.

We may begin with the MPIR for all new users—simply show them *something* to get them started and then learn as you go. But is there something better?

One approach you've probably experienced is the user onboarding flow: a simple set of questions employed by many websites to quickly ascertain basic information about the user, to help guide early recommendation. If discussing our book recommender, this might be asking what genres the user likes, or in the case of a coffee recommender, how the user brews coffee in the morning. It's probably clear that these questions are building up knowledge-based recommender systems and don't directly feed into our previous pipelines but can still provide some help in early recommendations.

If instead we looked at all our previous data and asked, "Which books in particular are most useful for determining a user's taste?," this would be an active learning approach. We could even have a decision tree of possibilities as the user answered each question, wherein the answer determines which next question is most useful to ask.

Summary

Now we have the confidence that we can serve up our recommendations, and even better, we have instrumented our system to gather feedback. We've shown how you can gain confidence before you deploy and how you can experiment with new models or solutions. Ensembles and cascades allow you to combine testing with iteration, and the data flywheel provides a powerful mechanism for improving your product.

You may be wondering how to put all this new knowledge into practice, to which the next chapter will speak. Let's understand how data processing and simple counting can lead to an effective—and useful!—recommendation system.